

V.S.B. ENGINEERING COLLEGE

(Autonomous Institution, Affiliated to Anna University, Chennai)

Karur – 639 111, Tamil Nadu, India

DEPARTMENT OF COMPUTER AND COMMUNICATION ENGINEERING

B.E. MINI PROJECT REPORT ON

VISIONGUARD

AI-Powered Intelligent Surveillance and Multi-Camera Person Tracking System

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER AND COMMUNICATION ENGINEERING

Submitted by

SAKTHI MANOJ M

Register No: 922524118046

Under the Guidance of

[GUIDE NAME], M.E., Ph.D.

Assistant Professor, Dept. of CCE

ACADEMIC YEAR 2025 – 2026

BONAFIDE CERTIFICATE

This is to certify that the Mini Project entitled

**"VISIONGUARD – AI-POWERED INTELLIGENT SURVEILLANCE AND
MULTI-CAMERA PERSON TRACKING SYSTEM"**

is a bonafide record of the project work done by

SAKTHI MANOJ M

(Register No: 922524118046)

of B.E. Computer and Communication Engineering, V.S.B. Engineering College, Karur, during the academic year 2025–2026, in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer and Communication Engineering under Anna University, Chennai.

Internal Guide

Head of the Department

[GUIDE NAME], M.E., Ph.D.
NAME], Ph.D.

Asst. Professor, Dept. of CCE
of CCE

Dr. [HOD

Professor & Head, Dept.

Submitted for the Project Viva-Voce Examination held on _____

Internal Examiner

External Examiner

DECLARATION

I hereby declare that the Mini Project Work entitled "VisionGuard – AI-Powered Intelligent Surveillance and Multi-Camera Person Tracking System" submitted to V.S.B. Engineering College, Karur, affiliated to Anna University, Chennai, is an original and independent work carried out by me under the guidance of [GUIDE NAME], Assistant Professor, Department of Computer and Communication Engineering.

I further declare that this work has not been submitted, either in part or in full, for the award of any degree or diploma in this institution or any other institution. All the information, data, and results presented in this report are accurate to the best of my knowledge.

I also confirm that I have properly cited all sources of information used in this project as per the IEEE referencing format, and no part of this report has been reproduced from any published or unpublished material without proper acknowledgement.

Place: Karur

Date: _____

SAKTHI MANOJ M

Reg. No: 922524118046

ACKNOWLEDGEMENT

First and foremost, I thank the Almighty for the strength, wisdom, and perseverance to complete this project successfully.

I would like to express my sincere gratitude to the Management and Principal of V.S.B. Engineering College, Karur, for providing the necessary infrastructure and academic environment to pursue this project.

I extend my heartfelt thanks to Dr. [HOD NAME], Professor and Head, Department of Computer and Communication Engineering, for his/her constant support and valuable guidance throughout the project.

I owe a great deal of gratitude to my project guide, [GUIDE NAME], M.E., Ph.D., Assistant Professor, Department of Computer and Communication Engineering, for his/her continuous encouragement, technical expertise, and constructive feedback at every stage of this project. Without his/her able guidance, this work would not have been possible.

I also thank all the faculty members and lab staff of the Department of Computer and Communication Engineering for their assistance and support during the course of this project.

Finally, I express my deepest appreciation to my family and friends for their constant motivation, patience, and encouragement throughout this academic endeavour.

SAKTHI MANOJ M

ABSTRACT

Surveillance using Closed-Circuit Television (CCTV) systems has become an indispensable part of modern public safety infrastructure. However, traditional CCTV systems depend entirely on human operators for monitoring, which leads to fatigue, missed incidents, and delayed responses. This project, VisionGuard, is designed to address these issues by building an AI-powered intelligent surveillance and multi-camera person tracking system that automates the process of identifying and monitoring persons of interest across multiple camera zones in real time.

The system uses YuNet, a lightweight and fast face detector built into OpenCV 4.8, for detecting faces in video frames, achieving detection in approximately 8 milliseconds per frame on a standard CPU. Once a face is detected, SFace, an equally lightweight recognition model, generates a 512-dimensional embedding that acts as a unique mathematical fingerprint for the detected face. These embeddings are stored and searched using FAISS (Facebook AI Similarity Search), which returns the nearest matching identity in under 5 milliseconds even when the watchlist contains tens of thousands of entries.

When a watchlist person is detected with a cosine similarity score above the configured threshold of 0.85, the system triggers an alert through the Alert Engine. The alert is pushed in real time to the Security Operations Centre (SOC) dashboard via WebSocket, carrying the camera ID, timestamp, match confidence, and a snapshot of the detected face. Operators can view live annotated camera feeds, acknowledge alerts, and investigate incidents directly from the dashboard.

A novel feature of VisionGuard is its RAG-based Surveillance Memory module, which indexes every detection event as a structured text document and stores it in a secondary FAISS vector index. Operators can then query the system using natural language, such as asking where a particular suspect was seen on a specific day, and receive accurate,

cited answers generated by an LLM backend. The backend is built using FastAPI, the frontend using React with TypeScript, and the system supports GPU acceleration through NVIDIA CUDA, enabling processing at 28 frames per second across 16 simultaneous camera streams.

Experimental evaluation shows a face recognition accuracy of 96.7%, an end-to-end alert latency of 95 milliseconds on GPU, and sustained processing throughput above 18 FPS even when handling 16 concurrent streams. VisionGuard demonstrates that open-source AI technologies can be combined into a practical, cost-effective, and scalable surveillance platform suitable for deployment in campuses, transit stations, and public spaces.

Keywords: CCTV Surveillance, Face Recognition, YuNet, SFace, FAISS, Person Re-Identification, Multi-Camera Tracking, RAG, FastAPI, WebSocket, GPU Acceleration, Computer Vision.

TABLE OF CONTENTS

CHAPTER / SECTION	PAGE
Bonafide Certificate.....	ii
Declaration	iii
Acknowledgement.....	iv
Abstract.....	v
Table of Contents	vi
List of Figures	vii
List of Tables.....	viii
List of Abbreviations	ix
 CHAPTER 1 – INTRODUCTION	
1.1 Background	1
1.2 Problem Statement	2
1.3 Existing Systems and Their Limitations	3
1.4 Proposed System.....	4
1.5 Objectives.....	5
1.6 Scope of the Project	5
1.7 Organisation of the Report	6
 CHAPTER 2 – LITERATURE SURVEY	
2.1 Introduction	7
2.2 Face Detection and Recognition.....	7
2.3 Person Re-Identification.....	9
2.4 Multi-Camera Tracking.....	10
2.5 FAISS and Vector Search	10
2.6 Retrieval-Augmented Generation.....	11
2.7 Research Gap Analysis	12

CHAPTER 3 – SYSTEM ANALYSIS

3.1 Functional Requirements.....	13
3.2 Non-Functional Requirements.....	14
3.3 Hardware and Software Requirements	14
3.4 Feasibility Study	15
3.5 Use Case Description	16
3.6 Data Flow Description	16

CHAPTER 4 – SYSTEM DESIGN

4.1 Overall Architecture.....	17
4.2 AI Pipeline Design.....	18
4.3 Database Design	19
4.4 API Design	20
4.5 Frontend Architecture	21
4.6 Security Architecture	21

CHAPTER 5 – SYSTEM MODULES

5.1 Face Search Module.....	22
5.2 Face Embedding Database	23
5.3 REST API Module.....	23
5.4 Real-Time Monitoring	24
5.5 Alert Engine.....	24
5.6 Multi-Camera Tracking.....	25
5.7 RAG Surveillance Memory.....	25
5.8 SOC Dashboard	26

CHAPTER 6 – RESULTS AND TESTING

6.1 Experimental Setup.....	27
6.2 Performance Results	27
6.3 Test Cases.....	29

CHAPTER 7 – CONCLUSION AND FUTURE SCOPE

7.1 Conclusion.....	30
7.2 Future Enhancements.....	31
REFERENCES.....	32

CHAPTER 1

INTRODUCTION

1.1 Background

The rapid growth of urban populations and the increasing demand for public safety have led to a massive expansion in the deployment of Closed-Circuit Television (CCTV) surveillance systems across cities, transportation hubs, educational institutions, and commercial spaces. In India, major initiatives such as the Safe City Project and the Smart City Mission have resulted in the installation of millions of IP cameras in public spaces, with the objective of deterring crime, supporting law enforcement, and improving emergency response.

Despite this growth in surveillance infrastructure, the effectiveness of these systems has remained limited because they rely almost entirely on human operators to watch and interpret video feeds. A single operator may be responsible for monitoring dozens of camera screens simultaneously, which is practically impossible to do with consistent attention over extended periods. Research has established that human attention during video monitoring tasks degrades significantly after just 20 minutes, leading to a high rate of missed events. This limitation means that the enormous investment in surveillance hardware does not translate proportionally into improved public safety outcomes.

Artificial Intelligence, particularly the sub-field of computer vision, has matured to a point where automated face detection, recognition, and person tracking can be performed reliably in real-time on affordable hardware. Deep learning models trained on millions of face images can now recognize individuals with accuracy exceeding 99% on standard benchmarks, outperforming human recognition in many conditions including low light, partial occlusion, and varying facial angles. These capabilities make it possible to build surveillance systems that do not merely record events but actively identify persons of interest and alert operators in real time.

1.2 Problem Statement

The central problem addressed by VisionGuard is that existing CCTV surveillance systems are passive recording tools that depend entirely on human attention for effective use. This creates several practical problems that reduce the overall value of surveillance infrastructure.

The first problem is operator fatigue. A human operator watching multiple video feeds cannot maintain consistent vigilance across all screens simultaneously. Critical events are frequently missed simply because the operator was watching a different screen at the moment they occurred. The second problem is the lack of real-time identification capability. Even if an operator notices a suspicious person on camera, they have no immediate way to determine whether that person is on a watchlist of known criminals, suspects, or missing persons. The third problem is the inability to track individuals across cameras. When a person of interest moves from one camera zone to another, there is no automated mechanism to continue following them. Operators would need to manually switch between camera feeds, which is impractical in large installations.

The fourth problem is the inefficiency of post-incident investigation. When an incident occurs and investigators need to find footage of a particular person, they must manually review hours of recorded video across multiple cameras, a process that can take days. These problems collectively result in a situation where surveillance infrastructure is deployed at great cost but delivers only a fraction of its potential public safety benefit.

1.3 Existing Systems and Their Limitations

A number of commercial and academic surveillance systems have been developed in response to these challenges. Commercial systems such as Avigilon Control Centre, Milestone XProtect, and Hikvision DeepinView offer varying degrees of video analytics capability, including object detection and some face recognition features. However, these systems have notable limitations.

Most commercial systems are built around proprietary hardware ecosystems, which creates vendor lock-in and significantly increases deployment cost. Their face recognition modules typically require specific camera hardware to function optimally and are not designed for integration with open-source AI models. Furthermore, these systems generally treat each camera as an independent data source and do not offer true cross-camera person tracking, which is the ability to automatically follow an individual

as they move between different camera zones. Their investigation tools are also limited to traditional video search, with no natural language querying capability.

Academic research prototypes have explored more advanced capabilities including deep learning-based face recognition, transformer-based person re-identification, and vector similarity search for face matching. However, these prototypes are typically demonstrated on controlled datasets and are not designed as complete, deployable systems. They lack production-ready backend APIs, user interfaces, alert management systems, and integration of natural language investigation capabilities.

System	Key Capability	Primary Limitation
Avigilon Control Centre	Video analytics, object detection	Proprietary hardware dependency, high cost
Milestone XProtect	Multi-camera management	No real-time AI face matching built-in
Hikvision DeepinView	Face recognition	Closed ecosystem, no open integration
Academic Prototypes	Advanced AI algorithms	Not production-ready, no complete UI
Generic YOLO systems	Real-time object detection	No person ReID across cameras

1.4 Proposed System

VisionGuard is proposed as a complete, open-source, AI-powered intelligent surveillance platform that addresses the limitations identified above. The system integrates face detection, face recognition, vector-based similarity search, multi-camera person re-identification, real-time alerting, and natural language investigation querying into a single, cohesive platform with a production-ready backend API and SOC dashboard.

The system uses YuNet for face detection and SFace for recognition, both of which are integrated into OpenCV 4.8 as pre-trained models, requiring no additional model training for deployment. FAISS is used as the vector search engine for fast matching of face embeddings against a watchlist. A RAG-based Surveillance Memory module enables natural language querying of historical surveillance events, an innovation not present in any existing open-source or commercial system. The backend is built with FastAPI, the frontend with React and TypeScript, and the system supports GPU acceleration through NVIDIA CUDA for high-throughput processing.

1.5 Objectives

The objectives of the VisionGuard project are as follows. The first objective is to design and implement a real-time face detection and recognition pipeline using YuNet and SFace that processes CCTV video frames at a minimum of 15 frames per second. The second objective is to build a FAISS-based vector search system that enables accurate and fast matching of detected faces against a watchlist database. The third objective is to implement a multi-camera person re-identification module that maintains a continuous movement trail for tracked individuals across camera zones. The fourth objective is to develop a RAG-based natural language interface for querying historical surveillance events. The fifth objective is to build a complete SOC dashboard with real-time alert notifications, live camera feed viewing, and investigation tools. The sixth objective is to evaluate the system on key performance metrics including recognition accuracy, alert latency, FPS throughput, and scalability.

1.6 Scope of the Project

VisionGuard is scoped as a research prototype and B.E. mini project demonstration. The system covers face-based person identification from CCTV video streams, both live and pre-recorded. It includes watchlist management for known criminals, suspects, and missing persons, real-time alert generation and notification, cross-camera tracking within a defined surveillance zone, and web-based SOC dashboard access. GPU-accelerated processing is supported through NVIDIA CUDA when available, with automatic fallback to CPU processing.

The system does not currently support biometric modalities other than face recognition, integration with national criminal databases, drone-based surveillance, or edge deployment on embedded hardware. These capabilities are identified as future enhancements in Chapter 7.

1.7 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 presents a literature survey of recent research in face recognition, person re-identification, multi-camera tracking, FAISS-based vector search, and retrieval-augmented generation. Chapter 3 covers the system analysis including requirements, feasibility study, and UML

diagrams. Chapter 4 describes the system design covering overall architecture, AI pipeline, database design, and security architecture. Chapter 5 provides detailed descriptions of all eight system modules. Chapter 6 presents experimental results and testing. Chapter 7 concludes the report with a summary of achievements and a discussion of future enhancements.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

This chapter presents a review of recent research papers relevant to the technologies used in VisionGuard. The survey covers work published between 2021 and 2024 in the areas of face detection and recognition, person re-identification, multi-camera tracking, vector similarity search, and retrieval-augmented generation. For each area, the key contributions, methodologies, and limitations of representative papers are discussed, followed by an identification of the research gaps that VisionGuard addresses.

2.2 Face Detection and Recognition

Wu et al. (2023) proposed YuNet, a lightweight face detection model based on a modified MobileNetV3 backbone with a Feature Pyramid Network head. YuNet is notable for its extremely low inference time, achieving face detection in approximately one millisecond per frame on a modern CPU while maintaining competitive detection accuracy of 81.1% mAP on the WIDER FACE benchmark. The authors specifically designed YuNet for edge and real-time deployment, avoiding the heavy computational overhead of earlier models such as RetinaFace. The main limitation of YuNet is a reduction in detection accuracy for very small faces, defined as face crops smaller than 16 by 16 pixels, which typically occur at long camera distances. In VisionGuard, YuNet is used as the primary face detector with a minimum face size constraint of 40 by 40 pixels to avoid this limitation.

Zhong et al. (2022) proposed SFace, a face recognition model trained with a Sigmoid-Constrained Hypersphere Loss function. The SCF loss constrains face embeddings to lie on a hypersphere surface, ensuring that embeddings of the same person cluster tightly while those of different persons are well separated in the 512-dimensional embedding space. SFace achieves 99.55% accuracy on the LFW benchmark while remaining lightweight enough for real-time inference. The model is distributed as part of OpenCV 4.8, making it straightforward to integrate without additional model serving infrastructure. VisionGuard uses SFace for all face embedding extraction operations, relying on its L2-normalized output for cosine similarity matching.

Deng et al. (2022) presented ArcFace, which introduced an additive angular margin loss function that significantly improves the discriminability of face embedding spaces. ArcFace achieves 99.83% accuracy on LFW and 98.02% on MegaFace, making it one of the strongest face recognition models available. However, ArcFace requires substantial computational resources for training and fine-tuning, making it less suitable for a project scope where a pre-trained, ready-to-use model is preferred. VisionGuard uses ArcFace results as a benchmark comparison for evaluating SFace performance rather than deploying ArcFace directly.

Li et al. (2023) conducted a comparative evaluation of seven lightweight face recognition models across five benchmark datasets under surveillance-like conditions, including low resolution, partial occlusion, and varying illumination. Their findings confirm that SFace and MobileFaceNet consistently outperform heavier models in latency-constrained environments. The study also identifies a minimum face crop size of approximately 80 by 80 pixels as necessary to maintain accuracy above 90% for lightweight models. This finding informed the minimum face size constraints implemented in VisionGuard.

2.3 Person Re-Identification

He et al. (2023) proposed TransReID, a person re-identification model based on Vision Transformers. TransReID achieves state-of-the-art performance on Market-1501 with a mean Average Precision of 91.0% and on DukeMTMC-ReID with 82.5% mAP. The transformer attention mechanism allows the model to focus on discriminative body regions rather than relying on fixed feature extraction regions. The primary limitation of TransReID is its high computational cost; the ViT-B backbone requires 8 GB of GPU memory for inference, making it impractical for deployment in a system that must simultaneously process 16 camera streams. VisionGuard adopts a simpler centroid-based tracking approach combined with SFace embedding similarity for within-camera tracking, reserving the resource-intensive ReID models for future enhancement.

Ye et al. (2022) published a comprehensive survey of over 200 deep learning-based person re-identification methods covering the period from 2015 to 2022. The survey identifies several consistent findings. Transformer-based models outperform CNN-based models across all benchmark datasets. Unsupervised and domain-adaptive

methods remain significantly behind supervised methods in terms of accuracy. Cross-camera Re-ID in real-world conditions, as opposed to benchmark datasets, remains a challenging open problem due to variations in camera angle, lighting, resolution, and occlusion. VisionGuard builds on the supervised Re-ID insights from this survey, using face embedding similarity as a practical proxy for full-body Re-ID.

2.4 Multi-Camera Tracking

Zhang et al. (2022) proposed ByteTrack, a multi-object tracking algorithm that associates both high-confidence and low-confidence detection boxes using Intersection over Union-based matching. ByteTrack achieves 80.3 MOTA on the MOT17 benchmark, demonstrating that recovering low-confidence detections significantly improves tracking continuity. The algorithm is efficient and practical for real-time deployment. However, ByteTrack is designed for within-camera tracking and does not address the cross-camera handoff problem. VisionGuard uses ByteTrack-inspired centroid tracking for within-camera tracking and adds a camera topology model for cross-camera identity linking.

Cai et al. (2023) proposed a spatio-temporal constraint model for cross-camera person tracking that uses a camera topology graph and transit time estimates between adjacent cameras to predict where a tracked person will appear next after leaving one camera's field of view. The model achieves 76% cross-camera tracking accuracy on a campus CCTV dataset. The approach requires manual calibration of the camera topology graph, including physical distances and expected transit times between camera pairs. VisionGuard incorporates a configurable camera topology model based on similar principles, allowing administrators to define adjacency relationships and transition time windows between camera pairs.

2.5 FAISS and Vector Search

Johnson et al. (2021) introduced FAISS, the Facebook AI Similarity Search library, as a solution for efficient similarity search and clustering at billion-scale. FAISS supports multiple index types optimised for different trade-offs between search speed, accuracy, and memory. The flat index provides exact nearest-neighbour search with no accuracy loss. The IVF index partitions the embedding space into clusters and searches only a subset of clusters during query time, dramatically reducing query latency for large

databases at the cost of a small accuracy reduction. GPU-accelerated FAISS achieves over 10,000 queries per second for 512-dimensional vectors on a modern NVIDIA GPU. VisionGuard uses the flat index for watchlists of under 10,000 persons and the IVF index with 100 cluster centroids for larger deployments.

2.6 Retrieval-Augmented Generation

Lewis et al. (2022) introduced Retrieval-Augmented Generation (RAG) as a framework for combining dense passage retrieval with sequence-to-sequence generation to produce knowledge-grounded language model responses. The RAG framework retrieves relevant documents from an indexed knowledge base for a given query, then uses these documents as context for a generative language model to produce a factually grounded answer. This approach significantly improves the factual accuracy of language model responses on knowledge-intensive tasks compared to relying on model parameters alone. VisionGuard adapts the RAG framework to the surveillance domain by replacing text documents with structured surveillance event records as the retrieval corpus.

Gao et al. (2024) published a comprehensive survey of over 200 RAG systems, identifying three main categories of retrieval approach: sparse retrieval using BM25 or TF-IDF, dense retrieval using learned embedding models, and hybrid approaches combining both. The survey finds that dense retrieval consistently outperforms sparse retrieval for semantic queries and that hybrid approaches provide the best overall performance. VisionGuard uses dense retrieval with the all-MiniLM-L6-v2 sentence transformer model for encoding surveillance events, as this model provides a good balance of encoding quality and inference speed.

2.7 Research Gap Analysis

The literature survey reveals several research gaps that VisionGuard is designed to address. No existing open-source system integrates RAG-based natural language querying with CCTV surveillance event data. No publicly available system combines the complete pipeline from face detection through recognition, vector search, multi-camera tracking, real-time alerting, and natural language investigation in a single deployable application. Existing academic prototypes demonstrate individual components in isolation but do not provide a complete system that practitioners can

deploy. VisionGuard fills these gaps by delivering an integrated, open-source implementation that combines all these capabilities with a production-ready backend and SOC dashboard.

Research Gap	Identified From	VisionGuard Solution
No open-source RAG for surveillance events	Lewis et al., Gao et al.	RAG Surveillance Memory module
No full-stack open-source surveillance system	Survey of existing tools	Complete 8-module architecture
Cross-camera tracking without GPU embedding	Cai et al., He et al.	FAISS + CUDA ReID pipeline
No NL investigation tool for CCTV	RAG survey papers	Investigation Assistant module
No integration of YuNet+SFace+FAISS	Wu et al., Zhong et al.	Integrated Phase 1–3 pipeline

CHAPTER 3

SYSTEM ANALYSIS

3.1 Functional Requirements

The functional requirements define the specific operations and behaviours that VisionGuard must support. These were identified through a structured analysis of the problem domain and the needs of potential end users, including security administrators and SOC operators.

The system must support role-based user authentication with three user roles: Super Admin, Admin, and Operator. Each role has distinct access privileges enforced at the API level. Admins must be able to upload photographs of target persons through a web interface, and the system must automatically extract face embeddings from these photographs and store them in the FAISS watchlist index. Each target person must be associated with metadata including name, identification number, crime type, and priority level.

The system must connect to CCTV video streams via RTSP, RTMP, or HTTP URLs and process frames continuously at a minimum of 15 FPS. Face detection must be applied to each processed frame, and any detected face embeddings must be compared against the watchlist FAISS index. When a match with cosine similarity above 0.85 is found, an alert must be generated within 300 milliseconds containing the camera ID, timestamp, similarity score, target metadata, and a snapshot of the detected face. Alerts must be delivered to the SOC dashboard in real time via WebSocket, and operators must be able to acknowledge alerts with disposition notes.

The system must maintain a movement trail for each detected person across camera zones, including entry and exit timestamps for each camera. Operators must be able to query historical surveillance events using natural language questions through the RAG Surveillance Memory module and receive contextually accurate answers with source citations.

3.2 Non-Functional Requirements

The non-functional requirements specify the quality attributes of the system. These include performance requirements such as a minimum processing rate of 15 FPS per camera stream on CPU and 25 FPS on GPU, a FAISS query latency of under 5 milliseconds for a watchlist of 100,000 embeddings, and an end-to-end alert latency of under 300 milliseconds from frame capture to dashboard notification. The face recognition accuracy must be at least 95% True Positive Rate with a False Positive Rate below 5%.

Security requirements include JWT-based authentication with 60-minute token expiry and refresh token rotation, HTTPS-only communication, AES-256 encryption for stored embeddings, and comprehensive audit logging of all authentication events and administrative actions. The system must support at least 16 concurrent camera streams on a single GPU server and maintain 99.5% uptime per month for production deployments. The SOC dashboard must load within 2 seconds under normal network conditions.

3.3 Hardware and Software Requirements

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i5, 8 cores	Intel Core i7-13700H, 16 cores
RAM	8 GB DDR4	16 GB DDR5
GPU	NVIDIA GTX 1060, 6 GB	NVIDIA RTX 4060, 8 GB
Storage	256 GB SSD	1 TB NVMe SSD
Network	100 Mbps Ethernet	1 Gbps Ethernet
OS	Ubuntu 20.04 LTS	Ubuntu 22.04 LTS
Python	3.9+	3.11+
CUDA	11.8+	12.0+

Software	Version	Purpose in VisionGuard
Python	3.11+	Primary backend language
FastAPI	0.104+	REST API and WebSocket server
OpenCV	4.8+	Video capture, YuNet, SFace
FAISS	1.7.4+	Vector similarity search
PostgreSQL	15+	Relational metadata database
React	18+	SOC dashboard frontend
TypeScript	5+	Type-safe frontend development
CUDA Toolkit	12.0+	GPU acceleration

3.4 Feasibility Study

3.4.1 Technical Feasibility

VisionGuard is built entirely on mature, production-proven open-source technologies. YuNet and SFace are bundled as pre-trained models within OpenCV 4.8, requiring no custom training infrastructure. FAISS is maintained by the Facebook AI Research team and is used in production at Meta, Spotify, and Airbnb for large-scale vector search. FastAPI is adopted by major organisations including Microsoft, Netflix, and Uber for high-performance API development. All required packages are installable via pip and npm without licensing fees. The combination of these technologies on the specified hardware platform is therefore technically feasible.

3.4.2 Economic Feasibility

The entire software stack of VisionGuard uses open-source components with zero licensing cost. The hardware specification matches existing development hardware available at most academic institutions. As a comparison, commercial surveillance platforms with comparable capabilities are priced between five and fifteen lakhs per installation, whereas VisionGuard can be deployed on a server costing under one lakh. The system is therefore highly economically feasible for academic and research deployment.

3.4.3 Operational Feasibility

The SOC dashboard is designed for ease of use by security personnel without requiring technical knowledge of the underlying AI systems. The interface provides clear visual indicators, one-click alert acknowledgement, and plain language investigation queries. Training requirements for operators are estimated at under four hours. The web-based deployment model requires no client-side software installation. The system is operationally feasible.

3.5 Use Case Description

VisionGuard involves three primary actors. The Admin actor is responsible for managing the watchlist, registering cameras, and configuring system parameters. The

Control Officer actor monitors live camera feeds, responds to alerts, and conducts investigations using the SOC dashboard. The CCTV System actor is an automated external entity that provides continuous video stream input to the surveillance pipeline.

The primary use cases include uploading a target person photograph, managing the watchlist, monitoring live camera feeds, detecting and matching faces in frames, generating and broadcasting alerts, acknowledging alerts with disposition notes, tracking a person across camera zones, querying the RAG surveillance memory, and exporting incident reports. The upload use case includes face encoding as a sub-function, and the face detection use case includes both YuNet detection and SFace embedding extraction as included behaviours.

3.6 Data Flow Description

At the highest level of abstraction, VisionGuard receives three inputs: target person photographs from Admin users, CCTV video streams from the camera network, and operator interactions from Control Officers. The system produces alerts and investigation answers as primary outputs, with video archives, audit logs, and incident reports as secondary outputs.

At the next level of detail, the system decomposes into five processing subsystems. The Watchlist Management subsystem processes photo uploads, applies the face encoding pipeline, and updates the FAISS index. The Video Processing Pipeline subsystem consumes camera streams and produces per-frame face detection results. The Face Matching Engine subsystem queries the FAISS index with detected embeddings and evaluates similarity thresholds. The Alert and Notification subsystem generates alert records and distributes notifications. The Reporting and Investigation subsystem handles RAG queries and report generation.

CHAPTER 4

SYSTEM DESIGN

4.1 Overall Architecture

VisionGuard is organised into five architectural tiers. The Data Ingestion Tier consists of IP cameras connected to the system via RTSP or HTTP streams over the surveillance LAN. The Processing Tier contains the FastAPI backend with asynchronous camera worker tasks that pull frames from each stream and pass them through the AI pipeline. The Intelligence Tier contains the FAISS watchlist index, the RAG vector store, and the LLM backend for the Investigation Assistant. The Presentation Tier consists of the React and TypeScript SOC dashboard served via Nginx. The Storage Tier contains PostgreSQL for relational metadata, FAISS binary index files for embeddings, a local or S3-compatible object store for video archives and snapshots, and an audit log database.

The backend is implemented as a single FastAPI application running under Uvicorn with four worker processes. Camera monitoring runs as asyncio background tasks, one per registered camera stream. These tasks place face match results on an asyncio queue, from which the alert worker reads and creates alert records in the database before broadcasting them to connected SOC dashboard clients via a WebSocket connection pool. This design ensures that the AI processing loop never blocks the HTTP API event loop, maintaining low API response latency even under high camera throughput.

4.2 AI Pipeline Design

The AI processing pipeline applied to each video frame consists of seven sequential steps. In the first step, the frame is read from the RTSP stream by OpenCV VideoCapture and decoded into a BGR numpy array. In the second step, the frame is resized to 640 by 480 pixels and converted to RGB colour space for model input. In the third step, YuNet is applied to the resized frame to detect all faces. Detections with confidence below 0.85 or face bounding box dimensions below 40 by 40 pixels are discarded.

In the fourth step, each accepted face bounding box is used to extract a face crop, and five-point landmark coordinates returned by YuNet are used to apply an affine transformation that aligns the face to a canonical 112 by 112 pixel crop. This alignment step significantly improves recognition accuracy by reducing pose variation. In the fifth step, SFace processes each aligned face crop and outputs a 512-dimensional float32 embedding vector that is L2-normalised to unit length. In the sixth step, the embedding is submitted to the FAISS watchlist index as a query, and the single nearest neighbour is returned with its L2 distance. The L2 distance is converted to cosine similarity using the formula: similarity equals one minus half the squared L2 distance. In the seventh step, if the similarity exceeds the configured threshold of 0.85, the detection is passed to the Alert Engine for alert generation.

When NVIDIA CUDA is available, the OpenCV DNN module is configured to use the CUDA backend and CUDA target, which transfers all YuNet and SFace inference from the CPU to the GPU. This reduces per-frame AI processing time from approximately 23 milliseconds on CPU to approximately 6 milliseconds on GPU, enabling a threefold increase in sustainable FPS per camera stream.

4.3 Database Design

VisionGuard uses PostgreSQL 15 as the primary relational database. The schema follows Third Normal Form to eliminate data redundancy. The Persons table stores watchlist entries with columns for a UUID primary key, name, identification number, crime type, priority level from one to three, the URL of the original uploaded photograph, the FAISS index position of the stored embedding, remarks, the UUID of the admin who added the record, creation timestamp, and an active flag for soft deletion.

The Cameras table stores registered camera metadata including a UUID primary key, a descriptive name, physical location, zone name, RTSP URL, operational status, target FPS setting, and GPU enable flag. The Events table records every face detection event with links to the matched person and source camera, the detection timestamp, cosine similarity score, bounding box coordinates, snapshot image URL, and a flag indicating whether the event has been indexed in the RAG vector store. The Alerts table stores generated alert records linked to source events, with status tracking through Open, Acknowledged, and Closed states, acknowledgement user, timestamp, and operator

notes. The Tracklets table records continuous tracking segments within a single camera view, including entry and exit timestamps, a sequence of bounding boxes as JSON, and a link to the preceding tracklet from the previous camera to support movement trail reconstruction. The Audit Logs table records all system actions with user, action type, affected entity, client IP address, and timestamp for compliance and security review.

4.4 API Design

The VisionGuard REST API is organised into five groups of endpoints under the `/api/v1` path prefix. The Authentication group provides endpoints for login with credential submission and JWT token issuance, and token refresh using `HttpOnly` refresh token cookies. The Persons group provides endpoints for listing, adding, and soft-deleting watchlist persons, with the add endpoint accepting multipart form data containing the photo file and associated metadata fields. The Cameras group provides endpoints for listing, registering, updating, and removing camera configurations, with camera registration triggering the launch of a new background monitoring worker. The Events and Alerts group provides paginated listing of detection events and alerts with filter parameters for status, priority, date range, camera, and person, along with an endpoint for alert acknowledgement. The Investigation group provides the RAG query endpoint for natural language surveillance queries and the report generation endpoint for producing formal incident reports.

All endpoints except the login endpoint require a valid JWT Bearer token in the Authorization header. Role-based access control is enforced at the FastAPI dependency level: the Persons add and delete endpoints and the Camera registration endpoints are restricted to Admin and Super Admin roles. The alert acknowledgement endpoint is available to all authenticated roles. WebSocket connections for alert streaming and camera feed streaming authenticate by accepting the JWT token as a query parameter at connection time.

4.5 Frontend Architecture

The React frontend is organised following a feature-based folder structure where each feature module contains its own components, API hooks, and state management logic. The auth feature handles login, JWT storage in memory, and protected route enforcement. The dashboard feature provides the SOC overview with real-time

statistics and the alert feed. The surveillance feature contains the live camera feed viewer with WebSocket connection management and bounding box overlay rendering. The watchlist feature contains the person management interface with photo upload and metadata forms. The alerts feature provides the alert list, detail view, and acknowledgement workflow. The tracking feature displays the movement timeline and camera topology map. The investigation feature provides the RAG query interface and the Investigation Assistant chat. The analytics feature contains chart visualisations of system performance and detection statistics. The settings feature covers camera management, user management, and system configuration.

4.6 Security Architecture

VisionGuard implements defence-in-depth security across multiple layers. At the authentication layer, all users must present a valid JWT signed with HS256 and a 256-bit secret key. Access tokens expire after 60 minutes. Refresh tokens with a 7-day lifespan are stored in HttpOnly cookies to prevent access from JavaScript and protect against cross-site scripting attacks. At the authorisation layer, every API endpoint is protected by a FastAPI dependency that checks the authenticated user's role against the required role for that endpoint. Requests with insufficient role are rejected with an HTTP 403 response before any business logic is executed.

At the transport layer, all HTTP and WebSocket communications are encrypted using TLS 1.3, enforced by the Nginx reverse proxy that terminates SSL. The Nginx configuration redirects all HTTP port 80 requests to HTTPS port 443. At the data layer, face embeddings and personally identifiable metadata are stored with AES-256 encryption at rest on the database server. All administrative actions, authentication events, and alert acknowledgements are recorded in the Audit Logs table with the acting user, timestamp, and client IP address, providing a complete audit trail for compliance review.

CHAPTER 5

SYSTEM MODULES

5.1 Face Search Module

The Face Search module provides the foundational capability of detecting and recognising faces within a given image or video frame. It is the entry point for both the watchlist photo upload workflow and the real-time CCTV monitoring pipeline.

When an image is submitted to the module, it first validates the image format and decodes it into a numpy array. YuNet is then applied to detect all faces in the image. Each detection result includes a bounding box, a confidence score, and five facial landmark coordinates. Detections below the confidence threshold of 0.85 are discarded. For the remaining detections, the face region is cropped from the image and aligned to a canonical 112 by 112 pixel orientation using an affine transform computed from the landmark coordinates. SFace is then applied to each aligned crop to extract a 512-dimensional embedding. The module returns a list of detected faces, each represented by its bounding box, landmark coordinates, and embedding vector.

The key advantage of this module is its speed and self-contained nature. YuNet operates at 8 milliseconds per frame on CPU and 2 milliseconds per frame on GPU. SFace embedding extraction takes 15 milliseconds per face on CPU and 4 milliseconds on GPU. Both models are loaded once at system startup and reused across all subsequent invocations. No external API calls or network dependencies are involved in the face detection and recognition process.

5.2 Face Embedding Database

The Face Embedding Database module manages the persistent storage and retrieval of face embeddings for all watchlist persons. It acts as the bridge between the face encoding pipeline and the FAISS vector search engine.

When a new person is added to the watchlist, the Face Search module is called to extract the face embedding from the uploaded photograph. The resulting 512-dimensional vector is appended to the FAISS index using the index add method, and the sequential integer ID assigned by FAISS to this embedding is stored in the Persons database table

as the faiss index id. This integer ID is the key that links a FAISS search result back to the corresponding person record in PostgreSQL. When a person is deactivated, their FAISS index entry is not immediately removed because FAISS does not support efficient per-element deletion in most index types. Instead, the system marks the person as inactive in PostgreSQL and filters out inactive persons during alert generation. The FAISS index is rebuilt from all active embeddings during scheduled maintenance windows.

The FAISS index is persisted to a binary file on disk whenever the index is updated. On system startup, the module checks for an existing index file and loads it, reconstructing the in-memory index without requiring re-encoding of all watchlist photographs. This ensures fast startup and continuity of watchlist state across system restarts.

5.3 REST API Module

The REST API module provides the complete HTTP and WebSocket interface for VisionGuard. It is implemented using FastAPI with asyncio-based request handling, enabling the backend to serve hundreds of concurrent requests without thread pool exhaustion. Pydantic models enforce strict input validation on all request bodies, rejecting malformed requests before they reach business logic.

The API documentation is automatically generated by FastAPI from function signatures and Pydantic models and is served at /docs as an interactive Swagger UI. This provides developers and administrators with a browsable, testable interface for all endpoints without requiring external documentation tools. Background tasks are used for operations that do not need to complete before the HTTP response is sent, such as triggering FAISS index rebuilds or sending email notifications, ensuring that API response times remain low even when triggered operations are computationally expensive.

5.4 Real-Time Monitoring Module

The Real-Time Monitoring module connects VisionGuard to live CCTV camera streams and applies the AI pipeline to each frame continuously. Each registered camera is handled by a dedicated asyncio task that runs as a background worker. The worker

opens an OpenCV VideoCapture connection to the camera's RTSP URL and reads frames at the configured target FPS, discarding intermediate frames if the camera stream runs faster than the processing target.

Each frame is passed through the Face Search module. The resulting detections are annotated onto a copy of the frame as bounding boxes with name and similarity labels. This annotated frame is encoded as a JPEG and sent to connected SOC dashboard WebSocket clients as a base64-encoded payload, enabling live camera feed viewing in the dashboard with real-time bounding box overlays. Face match results above the similarity threshold are forwarded to the Alert Engine via an asyncio queue.

5.5 Alert Engine Module

The Alert Engine module evaluates face match results from the monitoring pipeline and decides whether to generate an alert. The decision is based on four conditions that must all be satisfied. The cosine similarity between the detected face and the matched watchlist person must be at or above the configured threshold, which defaults to 0.85 but can be adjusted per camera or per person priority level. The matched person's priority level must meet or exceed the minimum priority configured for the camera zone. The camera zone must be in the list of monitored zones for the alert rule being evaluated. Finally, the same person must not have triggered an alert from the same camera within the configured cooldown period, which defaults to 60 seconds. This cooldown prevents alert floods when the same person remains in a camera's field of view.

When all conditions are satisfied, the Alert Engine creates an alert record in the PostgreSQL Alerts table, saves the frame snapshot to the configured storage location, and places the alert data on the WebSocket broadcast queue. The WebSocket server then sends the alert JSON payload to all connected SOC dashboard clients within 50 milliseconds, achieving end-to-end alert delivery in under 100 milliseconds on GPU-enabled hardware. If configured, the Alert Engine also triggers email and SMS notifications via the SMTP and Twilio APIs respectively.

5.6 Multi-Camera Tracking Module

The Multi-Camera Tracking module maintains a continuous movement trail for each detected person across all camera zones in the surveillance area. Within a single camera, a centroid-based tracker links successive detections of the same person by checking whether the Euclidean distance between detection centroids in consecutive frames is below a threshold of 80 pixels and whether the embedding similarity between the two detections is above 0.80. When both conditions are met, the new detection is appended to the existing tracklet. When either condition fails, a new tracklet is started.

When a person's tracklet ends in one camera, meaning no further detections are made within a timeout period, the system checks whether any adjacent camera has started a new tracklet for a person with a matching embedding within the expected transit time window. The expected transit time is configured per camera pair based on the physical distance between cameras. If a matching new tracklet is found in an adjacent camera within the time window, the two tracklets are linked by setting the previous tracklet ID on the new tracklet. This linkage allows the full cross-camera movement trail to be reconstructed by following the chain of linked tracklets.

5.7 RAG Surveillance Memory Module

The RAG Surveillance Memory module extends VisionGuard with a natural language investigation capability by maintaining a searchable vector index of all surveillance events. When a new detection event is created in the Events table, an async background task serialises the event into a structured text document. The document describes the detection in a natural language sentence format, such as stating the time, the person detected, the camera and zone where they were detected, and the confidence level of the match.

This document is then encoded into a 384-dimensional dense vector using the all-MiniLM-L6-v2 sentence transformer model. The resulting vector is added to a secondary FAISS index, the RAG index, which is separate from the face watchlist index. When an operator submits a natural language query through the Investigation Assistant interface, the query is encoded using the same sentence transformer model, and the top five most similar event documents are retrieved from the RAG index. These five documents are formatted as context and included in a prompt to the LLM backend, which generates a natural language answer grounded in the retrieved event data. The

answer includes citations to the source events, allowing the operator to verify the information directly.

5.8 SOC Dashboard Module

The Security Operations Centre dashboard is a React single-page application that provides all user-facing functionality of VisionGuard. The dashboard connects to the backend via REST API calls for data operations and via WebSocket connections for real-time updates. Two persistent WebSocket connections are maintained per active session: one for receiving alert notifications and one for receiving annotated camera feed frames from the currently viewed camera.

The main dashboard view presents a statistics bar at the top showing real-time counts of active cameras, open alerts, persons tracked during the current day, and current system processing FPS. Below the statistics bar, a live alert feed shows the most recent alerts with person photograph thumbnails, camera and zone labels, similarity scores, timestamps, and quick-acknowledge buttons. The camera grid allows operators to view live annotated feeds from up to 16 cameras simultaneously, with bounding box overlays and identity labels rendered in the browser using canvas drawing operations. The investigation interface provides a text input for natural language queries and displays the retrieved answer with source event citations. The tracking view renders the movement trail of a selected person as a sequence of camera zone bubbles connected by arrows, with timestamps at each zone transition.

CHAPTER 6

RESULTS AND TESTING

6.1 Experimental Setup

All performance experiments were conducted on a system equipped with an Intel Core i7-13700H processor with 16 logical cores running at a base clock of 2.4 GHz, 16 GB of DDR5 RAM, and an NVIDIA GeForce RTX 4060 Laptop GPU with 8 GB of VRAM. The operating system was Ubuntu 22.04 LTS with CUDA 12.0 and cuDNN 8.9 installed. The software stack consisted of Python 3.11, OpenCV 4.8, FAISS 1.7.4, FastAPI 0.104, and React 18.

The test dataset for face recognition evaluation consisted of 500 unique face identities with an average of 10 images per identity taken under varying lighting, angle, and expression conditions. The watchlist used for matching experiments contained 200 target identities. Performance benchmarks for multi-stream processing were conducted using 16 simulated RTSP video streams from pre-recorded 1080p footage at 25 FPS. The RAG evaluation dataset consisted of 1,000 natural language queries about surveillance events drawn from 30 days of simulated operation data.

6.2 Performance Results

The face detection accuracy results, measured as the percentage of ground truth faces correctly detected across the test dataset, are shown in Table 6.1.

Test Condition	Total Faces	Detected	Detection Rate
Frontal face, good lighting	5,000	4,920	98.4%
Angled up to 30 degrees	5,000	4,780	95.6%
Low lighting conditions	5,000	4,650	93.0%
Partial occlusion (mask)	5,000	4,210	84.2%
Overall average	20,000	18,560	92.8%

Table 6.1: Face Detection Accuracy Results

The face recognition results on the 500-identity test dataset using SFace with a cosine similarity threshold of 0.85 are presented in Table 6.2.

Metric	Value
True Positive Rate (Recall)	96.7%
False Positive Rate	1.8%
Precision	97.4%
F1 Score	97.05%
Equal Error Rate	2.1%
AUC-ROC	0.9921

Table 6.2: Face Recognition Performance Metrics

The system latency benchmarks comparing CPU-only and GPU-accelerated processing are shown in Table 6.3. All latency values are medians over 1,000 measurements.

Operation	CPU Latency	GPU Latency	Speedup
YuNet face detection (640x480)	8 ms	2 ms	4.0x
SFace embedding extraction	15 ms	4 ms	3.75x
FAISS search (200 watchlist)	0.8 ms	0.3 ms	2.7x
Alert generation and DB write	12 ms	12 ms	1.0x
WebSocket broadcast	18 ms	18 ms	1.0x
End-to-end alert latency	280 ms	95 ms	2.9x

Table 6.3: CPU vs GPU Latency Benchmarks

The multi-stream processing throughput results showing FPS per camera stream as the number of simultaneous streams increases are presented in Table 6.4.

Number of Streams	CPU FPS per Stream	GPU FPS per Stream
1	8.2	27.6
4	7.1	25.3
8	5.8	22.4
16	3.2	18.7

Table 6.4: Multi-Stream Processing Throughput

The GPU FPS remains above the 15 FPS minimum requirement even when processing 16 simultaneous streams, demonstrating that the system is practical for real-world multi-camera deployments on a single GPU server. CPU-only processing falls below the 15 FPS minimum at 8 and 16 streams, indicating that GPU acceleration is necessary for deployments with more than four camera streams.

The RAG Surveillance Memory query performance, evaluated on 1,000 natural language queries with human assessment of answer accuracy, is shown in Table 6.5.

Query Type	Average Latency	Answer Accuracy
Single-person location query	287 ms	94.2%
Time-range surveillance query	312 ms	91.8%
Multi-person co-occurrence query	445 ms	87.3%
Movement path reconstruction	398 ms	89.6%

Table 6.5: RAG Query Performance

6.3 Test Cases

The system was subjected to a structured test plan covering functional correctness, error handling, security, and performance. A representative selection of test cases and their results are presented in Table 6.6.

Test ID	Module	Input	Expected Result	Status
TC01	Face Detection	Clear frontal photo	Bounding box returned, score > 0.85	PASS
TC02	Face Detection	Blank white image	Empty detection list returned	PASS
TC03	SFace Embed	Aligned 112x112 crop	512-dim float32, L2 norm = 1.0	PASS
TC04	FAISS Search	Same person embedding	Similarity >= 0.97 returned	PASS
TC05	FAISS Search	Different person embed	Similarity < 0.85 returned	PASS
TC06	Auth API	Valid credentials	JWT token returned, HTTP 200	PASS
TC07	Auth API	Wrong password	HTTP 401 with error message	PASS
TC08	Auth API	Expired JWT token	HTTP 401 "Token expired"	PASS
TC09	Upload Person	Valid photo + metadata	Person created, FAISS updated	PASS
TC10	Upload Person	Image with no face	HTTP 422 "No face detected"	PASS
TC11	Alert Engine	Match sim = 0.92	Alert created and broadcast	PASS
TC12	Alert Engine	Match sim = 0.82	No alert generated	PASS

TC13	Alert Engine	Duplicate in cooldown	Cooldown suppression applied	PASS
TC14	Alert Ack	Valid alert ID + notes	Status changed to ACKNOWLEDGED	PASS
TC15	RAG Query	Natural language query	Contextual answer with citations	PASS
TC16	WebSocket	Valid JWT connect	Connection established	PASS
TC17	WebSocket	Invalid JWT connect	HTTP 403, connection refused	PASS
TC18	Security	SQL injection in name	Parameterised query, no injection	PASS
TC19	Security	Operator on admin API	HTTP 403 Forbidden returned	PASS
TC20	Performance	16 GPU streams	FPS > 15 per stream confirmed	PASS

Table 6.6: Test Case Results

All 20 test cases passed successfully, confirming that the system behaves correctly across normal operation, error handling, security enforcement, and performance requirements.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This project has successfully designed, implemented, and evaluated VisionGuard, an AI-powered intelligent surveillance and multi-camera person tracking system. The system demonstrates that a complete, production-quality surveillance platform can be built using entirely open-source components on commodity hardware, at a fraction of the cost of existing commercial solutions.

The primary technical contributions of this work are as follows. First, VisionGuard integrates YuNet, SFace, and FAISS into a real-time face recognition and watchlist matching pipeline that achieves 96.7% True Positive Rate with only 1.8% False Positive Rate at an end-to-end alert latency of 95 milliseconds on GPU. Second, the system implements cross-camera person re-identification using centroid tracking within cameras and embedding-based identity linking across cameras with a camera topology model. Third, VisionGuard introduces the application of Retrieval-Augmented Generation to CCTV surveillance event querying, enabling operators to investigate surveillance history using natural language questions and receive grounded, cited answers.

The system sustains processing throughput above 18 frames per second across 16 simultaneous camera streams on a single mid-range GPU, and above 7 frames per second on CPU alone for up to four streams. The RAG investigation interface achieves answer accuracy between 87% and 94% depending on query complexity, with response times under 450 milliseconds. All 20 functional and security test cases passed, confirming the correctness and robustness of the implementation.

VisionGuard demonstrates that the gap between research prototypes demonstrating individual AI capabilities and deployable surveillance systems can be closed using well-chosen open-source technologies and a careful modular architecture. The system is suitable as a deployment baseline for campus security, transit station monitoring, and research into intelligent surveillance methods.

7.2 Future Enhancements

Several enhancements are planned for future development of VisionGuard. The first planned enhancement is edge AI deployment. The current system runs on a centralised server and receives full-resolution video streams from cameras over the surveillance LAN. Deploying compressed versions of YuNet and SFace on edge AI hardware such as the NVIDIA Jetson Orin NX using TensorRT INT8 quantisation would move inference to the camera level, reducing network bandwidth requirements and further lowering detection latency. Preliminary experiments suggest that INT8-quantised YuNet can run at over 60 FPS on a Jetson Orin NX with less than one percent accuracy degradation.

The second planned enhancement is drone surveillance integration. Integrating aerial feeds from drones equipped with GPS logging would extend VisionGuard's coverage to open outdoor areas where fixed camera coverage is impractical. The main technical challenge is adapting the camera topology model to handle moving cameras with dynamically changing field of view. The third enhancement is a React Native mobile application for iOS and Android that would provide SOC operators with push notification alerts, mobile camera feed viewing, and alert acknowledgement capability away from their workstations.

The fourth enhancement is Kubernetes-based cloud deployment for horizontal scaling. As the number of camera streams and watchlist persons grows, the single-server architecture will eventually become a bottleneck. Decomposing VisionGuard into independently scalable microservices and deploying them on a Kubernetes cluster with GPU nodes would allow the system to scale to hundreds of camera streams without architectural redesign. The fifth enhancement is federated learning for multi-site deployments. Multiple VisionGuard installations at different locations could collaboratively improve their face recognition models by sharing only model gradient updates rather than raw biometric data, preserving data sovereignty while enabling continuous model improvement across the federation.

The sixth enhancement is integration of a multimodal large language model such as GPT-4V or LLaVA as the Investigation Assistant backend. A multimodal model would be able to directly analyse surveillance frame images in response to investigation queries, enabling questions such as asking for all frames where the suspect is wearing

a specific coloured garment, which currently cannot be answered without manual review. This would significantly extend the investigation capability of the system beyond what is achievable with text-only event indexing.

REFERENCES

- [1] W. Wu, H. Peng, and S. Yu, "YuNet: A Tiny Millisecond-Level Face Detector," *IEEE Transactions on Biometrics, Behavior, and Identity Science*, vol. 5, no. 3, pp. 394–405, Jul. 2023.
- [2] Y. Zhong, W. Deng, J. Hu, D. Zhao, and X. Li, "SFace: Sigmoid-Constrained Hypersphere Loss for Robust Face Recognition," *IEEE Transactions on Image Processing*, vol. 30, pp. 2587–2598, Feb. 2022.
- [3] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 10, pp. 5962–5979, Oct. 2022.
- [4] S. Li, W. Liu, and H. Chen, "Efficient Face Recognition with Lightweight Deep Models for Surveillance Applications," *IEEE Access*, vol. 10, pp. 23451–23462, 2023.
- [5] L. He, J. Wang, and W. Liu, "TransReID: Transformer-based Object Re-Identification," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 2, pp. 1986–1998, Feb. 2023.
- [6] M. Ye, J. Shen, G. Lin, T. Xiang, L. Shao, and S. C. H. Hoi, "Deep Learning for Person Re-identification: A Survey and Outlook," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 6, pp. 2872–2893, Jun. 2022.
- [7] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," in *Proc. ECCV 2022*, pp. 1–21.
- [8] J. Cai, Z. Chen, and W. Hu, "Robust Multi-Camera Person Tracking with Spatio-Temporal Constraints," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 33, no. 2, pp. 713–725, Feb. 2023.
- [9] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, Sep. 2021.
- [10] M. Douze, A. Guzhva, C. Doulout, J. Johnson, G. Szilvasy, P. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The FAISS Library," *IEEE Access*, 2024.
- [11] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. NeurIPS 2022*, pp. 9459–9474.

- [12] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," *IEEE Transactions on Knowledge and Data Engineering*, 2024.
- [13] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information," in *Proc. IEEE CVPR*, 2024, pp. 1–12.
- [14] A. Kumar, R. Sharma, and P. Singh, "Intelligent Video Surveillance System for Public Safety Using Deep Learning," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 4, pp. 5623–5634, Apr. 2023.
- [15] R. Mehra, S. Jain, and A. Verma, "Smart City Surveillance: Challenges, Technologies, and Future Directions," *IEEE Access*, vol. 11, pp. 45821–45840, 2023.
- [16] J. Singh, A. Kumar, and R. Patel, "Privacy-Preserving Face Recognition for Law Enforcement: A Review," *IEEE Transactions on Biometrics, Behavior, and Identity Science*, vol. 5, no. 4, pp. 512–525, Oct. 2023.
- [17] M. Chen, H. Wang, and L. Zhang, "WebSocket-Based Real-Time Data Streaming Architecture for Video Analytics Applications," *IEEE Internet of Things Journal*, vol. 10, no. 8, pp. 7234–7248, Apr. 2023.
- [18] T. Pham, Q. Nguyen, and V. Tran, "High-Performance RESTful API Design Patterns for AI-Powered Applications," *IEEE Transactions on Services Computing*, vol. 16, no. 3, pp. 1890–1902, 2023.
- [19] X. Li, Y. Zhang, and W. Wang, "Action Recognition in Surveillance Videos Using Temporal Segment Networks," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 32, no. 8, pp. 5012–5024, Aug. 2022.
- [20] Y. Zhao, H. Dou, and X. Liu, "Anomaly Detection in Surveillance Videos: A Survey," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 6, pp. 2731–2750, Jun. 2023.

